

TP Filtrage spatial Image

But du TP: Filtrage Linéaire

L'objectif de ce TP est de permettre à l'étudiant de :

- **Se familiariser avec la notion de convolution**, concept fondamental du traitement du signal et de l'image.
- **Apprendre à manipuler les différents noyaux** (kernels) des filtres linéaires.
- **Appliquer ces noyaux** pour réaliser diverses opérations de filtrage (lissage, détection de contours, etc.).

Types de bruits dans une image

En traitement d'images, le bruit correspond à des variations indésirables de la luminosité ou de la couleur. Voici les principaux types que vous rencontrerez dans votre TP, ainsi que leur implémentation avec la fonction `imnoise` de Matlab.

1) Bruit Gaussien (Gaussian Noise)

Définition

Bruit aléatoire suivant une loi normale (moyenne μ , variance σ^2).
Très fréquent lors de l'acquisition (capteurs, électronique).

Effet : Image légèrement granuleuse, variations continues d'intensité.

Implémentation MATLAB

```
I = imread('cameraman.tif');  
I = im2double(I);  
  
% Ajout bruit gaussien (moyenne=0, variance=0.01)  
J = imnoise(I, 'gaussian', 0, 0.01);  
  
figure;  
subplot(1,2,1), imshow(I), title('Image originale');  
subplot(1,2,2), imshow(J), title('Bruit gaussien');
```



2) Bruit Sel & Poivre (Salt & Pepper)

Définition : Pixels aléatoires remplacés par 0 (noir) ou 1 (blanc).

Effet : Points noirs et blancs isolés.

La densité dans le bruit **Salt & Pepper** représente : Pourcentage de pixels modifiés

Donc :

- 0.001 $\rightarrow$ 0.1 % des pixels sont modifiés
- 0.01 $\rightarrow$ 1 %
- 0.05 $\rightarrow$ 5 %

Implémentation MATLAB

```
I = imread('cameraman.tif');  
I = im2double(I);  
% Densité 5%  
J = imnoise(I, 'salt & pepper', 0.05);  
figure;  
subplot(1,2,1), imshow(I), title('Image originale');  
subplot(1,2,2), imshow(J), title('Bruit sel & poivre');
```



3) Bruit Poisson (Shot Noise)

Définition : Le bruit de Poisson (aussi appelé bruit de grenaille ou *shot noise*) est très particulier car, contrairement au bruit gaussien, il n'est pas ajouté uniformément sur toute l'image. Il est intrinsèquement lié à l'intensité de la lumière.

Effet : Le bruit de Poisson se manifeste par un scintillement ou un grain qui est plus fort dans les zones claires et plus faible dans les zones sombres.

- Dans les zones de haute intensité (blanches), les fluctuations sont plus grandes.
- Dans les zones de basse intensité (noires), le bruit est presque nul.

Cela crée une texture granuleuse qui semble "suivre" les formes de l'image, contrairement au bruit gaussien qui recouvre l'image comme un voile statique et uniforme.

Implémentation MATLAB

```
I = imread('cameraman.tif');  
I = im2double(I);  
J = imnoise(I, 'poisson');
```

```
figure;
subplot(1,2,1), imshow(I), title('Image originale');
subplot(1,2,2), imshow(J), title('Bruit Poisson');
```



4) Bruit Speckle (multiplicatif)

Définition : Bruit multiplicatif (souvent en imagerie radar ou échographie).

Effet : Le bruit de speckle donne à l'image un aspect extrêmement granuleux, comme si elle était composée de minuscules taches sombres et claires entremêlées.

- Contrairement au bruit Gaussien qui est "doux", le speckle est très "nerveux" et dégrade fortement la visibilité des petits détails.
- Comme il est multiplicatif, il est presque invisible dans les zones noires et très intense dans les zones claires.

Implémentation MATLAB

```
I = imread('cameraman.tif');
I = im2double(I);
J = imnoise(I, 'speckle', 0.04);
figure;
subplot(1,2,1), imshow(I), title('Image originale');
subplot(1,2,2), imshow(J), title('Bruit Speckle');
```



Evaluation des performances

1) MSE – Mean Squared Error (Erreur quadratique moyenne)

Définition

Mesure l'erreur moyenne entre l'image originale et l'image restaurée.

Interprétation

- Plus **MSE est petit** → meilleure qualité

MATLAB

`MSE = immse(I_filtre, I_originale);`

2) PSNR – Peak Signal to Noise Ratio

Définition

Rapport signal/bruit exprimé en décibels (dB).

$MAX = 1$ (si image double) ou 255 (si uint8)

Interprétation

- $PSNR > 30$ dB → bonne qualité
- $PSNR > 40$ dB → très bonne qualité
- Plus **PSNR est grand** → meilleure qualité

MATLAB

`PSNR = psnr(I_filtre, I_originale);`

3) SSIM – Structural Similarity Index

Définition

Mesure la similarité structurelle (luminance + contraste + structure).

Valeur entre :

- 0 → totalement différent
- 1 → identique

Avantage

Plus proche de la perception humaine que MSE.

MATLAB

`[SSIM_val, ssim_map] = ssim(I_filtre, I_originale);`

ssim_map est une matrice de la même taille que vos images d'entrée. Chaque pixel de cette carte a une valeur comprise entre -1 et 1 :

- **Proche de 1** : Les deux images sont presque identiques dans cette zone.

- **Proche de 0 ou négatif** : Les deux images sont très différentes (perte de structure, flou important ou bruit résiduel).

Le lien avec SSIM_val

La valeur SSIM_val que vous récupérez est simplement la moyenne de toutes les valeurs contenues dans la ssim_map :

4) SNR – Signal to Noise Ratio

Matlab

```
signal = sum(I_originale(:).^2);
noise = sum((I_originale(:)-I_filtre(:)).^2);
SNR = 10*log10(signal/noise);
```

Travail demandé :

1) Chargement et affichage de l'image : À l'aide des commandes `imread` et `imshow`, chargez et affichez l'image « **eight.tif** ».

2) Ajout de bruit : Ajouter un bruit Salt & Pepper à l'image de manière à obtenir un SNR de 20 dB. Déterminer expérimentalement la densité correspondante. Affichez l'image originale et l'image bruitée côte à côte dans une même figure.

3) Filtrage par convolution : Nous souhaitons maintenant filtrer l'image bruitée par une opération de convolution. Pour cela, appliquez trois filtres différents :

- Un filtre **moyenneur** ;
- Un filtre **gaussien** ;
- Un filtre **exponentiel**.

$\frac{1}{25} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$ <i>Filtre moyenneur (5x5)</i>	$\frac{1}{864} \begin{pmatrix} 11 & 23 & 29 & 23 & 11 \\ 23 & 48 & 62 & 48 & 23 \\ 29 & 62 & 80 & 62 & 29 \\ 23 & 48 & 62 & 48 & 23 \\ 11 & 23 & 29 & 23 & 11 \end{pmatrix}$ <i>Filtre gaussien ($\sigma = 1,41$)</i>	$\frac{1}{80} \begin{pmatrix} 1 & 1 & 3 & 1 & 1 \\ 1 & 3 & 7 & 3 & 1 \\ 3 & 7 & 16 & 7 & 3 \\ 1 & 3 & 7 & 3 & 1 \\ 1 & 1 & 3 & 1 & 1 \end{pmatrix}$ <i>Filtre exponentiel ($\gamma = 0,8$)</i>
---	---	--

3) Création des noyaux Créez dans Matlab® les noyaux de taille **5×5** pour les trois filtres définis précédemment (moyenneur, gaussien et exponentiel).

4) Filtrage par convolution : En utilisant la commande `imfilter`, calculez le produit de convolution de l'image bruitée avec chacun des trois noyaux. Affichez les résultats du filtrage ainsi que l'image originale et l'image bruitée dans une même figure pour faciliter la comparaison.

Interprétation :

Noyau moyenneur (fm)

- Simple moyenne de 5x5 pixels

- Efficace pour réduire le bruit mais floute les détails

Noyau Gaussien (fg)

- Pondération centrée sur le pixel central
- Préserve un peu mieux les contours

Noyau exponentiel (fe)

- Pondération plus forte au centre
- Compromis entre réduction de bruit et conservation des détails

imfilter(...,'replicate')

- Permet de gérer les bords en répétant les pixels

PSNR

- Permet de comparer quantitativement la qualité de chaque filtrage

Conclusion :

Moyenneur → bruit réduit mais image légèrement floue

Gaussien → bruit réduit, détails mieux conservés

Exponentiel → compromis : moins flou que le moyenneur, bruit réduit

- Plus le SNR augmente → meilleure qualité après filtrage

5) Filtrage Médian : Utilisez la commande `medfilt2` pour filtrer l'image bruitée avec un filtre médian. Ajoutez le résultat de ce filtrage à la figure précédente et comparez son efficacité par rapport aux filtres linéaires (moyenneur, gaussien, exponentiel).

Remarque : Bien que le filtre médian produise une amélioration visuelle significative pour le bruit impulsif, son SNR peut être inférieur à celui des filtres linéaires car le SNR mesure une erreur énergétique globale et non la qualité perceptuelle.

Filtre médian (non linéaire)

- ✓ Très efficace contre le bruit impulsif
- ✓ Préserve les contours
- ✓ Amélioration visuelle nette
- ✗ PSNR parfois inférieur aux filtres linéaires

Le plus adapté au bruit Salt & Pepper

Optimise la qualité perceptuelle plutôt que l'erreur quadratique

6) Analyse de robustesse Appliquez d'autres types de bruit à l'image originale.

7) Refaites le filtrage de ces nouvelles images bruitées en utilisant les quatre filtres définis précédemment. Concluez par vos remarques concernant la fiabilité et l'adéquation de chaque filtre selon le type de bruit appliqué.

- **Bruit Gaussien :**

Note : Effet du filtre médian

- ✗ Moins performant que les filtres linéaires

- ✗ PSNR souvent plus faible
- ✗ Peut dégrader inutilement les textures

Pourquoi ?

Le médian est conçu pour supprimer des valeurs extrêmes.
Or le bruit gaussien ne contient pas de valeurs aberrantes isolées.

Il ne correspond pas à la nature du bruit.

8) Détection de contours par filtrage spatial (Gradients)

- Comprendre le rôle des filtres de dérivation dans l'extraction de caractéristiques.
- Implémenter et comparer les masques de **Roberts**, **Prewitt** et **Sobel**.
- Analyser l'impact de la taille du noyau sur la sensibilité au bruit et la précision des contours.

Partie A : Initialisation

1. Chargez l'image cameraman.tif et convertissez-la en type double pour permettre les calculs de précision.
2. Pourquoi est-il nécessaire de travailler en double plutôt qu'en uint8 lors du calcul des gradients?

Partie B : Implémentation du filtre de Roberts (2×2)

1. Définissez les masques G_x et G_y de Roberts :
2. Utilisez la commande `imfilter` avec l'option 'replicate' pour calculer G_x et G_y .
3. Calculez le module du gradient G .
4. Affichez les trois résultats (G_x , G_y , G) dans une même figure.

Partie C : Extension aux filtres de Prewitt et Sobel (3×3)

1. Reproduisez la démarche précédente pour les filtres de **Prewitt** et **Sobel** en utilisant leurs masques respectifs.

Note : Le filtre de Sobel accorde un poids plus important (coefficient 2) au pixel central pour lisser le bruit.

2. Organisez l'affichage dans une figure globale de 9 sous-graphiques (`subplot(3,3,n)`) :
 - **Ligne 1** : Roberts (G_x , G_y , Module)
 - **Ligne 2** : Prewitt (G_x , G_y , Module)
 - **Ligne 3** : Sobel (G_x , G_y , Module)

Partie D : Analyse et comparaison

1. Observez les résultats. Quel filtre semble le plus sensible aux petits détails (et au bruit) ?
2. Lequel produit les contours les plus lisses et les plus continus ?

Quelques points d'analyse pour votre TP

- **Différence Horizontal/Vertical** : Vous remarquerez que les masques M_x (souvent appelés filtres verticaux) mettent en évidence les lignes verticales car ils calculent la différence d'intensité de gauche à droite. Inversement pour M_y .
- **Épaisseur des contours** : Les contours de **Robert** sont très fins (2x2) mais souvent "hachés" par le bruit. Les contours de **Sobel** et **Prewitt** (3x3) sont plus nets et continus.
- **Sensibilité** : **Sobel** est généralement plus "propre" que les autres car son coefficient **2** au centre lui donne un léger effet de lissage (filtre passe-bas intégré).